

Here's the final-calculation algorithm, based on the standard "Кодекс Преферанса" plus the common variants (Сочинка, Ростов, Классика, Ленинградка). The core method is called «амнистия + средняя гора» (or the equivalent «4 кружочка» method) — same idea, slightly different bookkeeping.

0. Base conversion rates (set by convention before play starts)

- 1 очко в горе = 10 вистов — this is nearly universal.
- 1 очко в пуле = 1 очко в горе (Сочинка, Ростов, Классика).
- **Exception — Ленинградка:** 1 очко в пуле = 2 очка в горе (everything doubles).

Step 1 — Уравнивание пуля (equalize the пуля)

Since players finish with slightly different totals in the "пуля" column (someone overshoots the target, e.g. 50, someone falls short), you convert this into ropa:

- **Сочинка/Ростов/Классика ("помощь"):** the player who overshoot the target subtracts the excess from his ropa; a player short of the target adds the shortfall to his ropa.
- **Ленинградка:** same idea, but the excess/shortfall is **multiplied by 2** before being applied to ropa (because 1 пуля-очко = 2 гора-очка here).

After this step, пуля no longer matters — everything lives in ropa and висты.

Step 2 — Амнистия (optional but standard convenience step)

Find the player with the **smallest** ropa value. Subtract that value from everyone's ropa (including that player's, making it 0). This doesn't change relative results, just makes the arithmetic easier — an "амнистёр" is created.

Step 3 — Convert гора into висты

There are two mathematically equivalent methods (both appear in the Кодекс, §4.2.14):

Вариант 1 (per-player division):

For each player: $\text{висты_проигрыша} = (\text{его гора} \times 10) / N$, where N = number of players (3 or 4). This is what that player owes to **each** other player individually, and gets added to their vist tallies against him.

Вариант 2 (средняя гора — "average gora" method):

1. Sum everyone's ropa and divide by $N \rightarrow$ **средняя гора**.
2. For each player: $(\text{средняя гора} - \text{его гора}) \times 10 =$ his net win/loss from the горка. Positive if his ropa is below average, negative if above.

Both give the same final numbers — Вариант 2 just does one round of subtraction+summation instead of $N-1$ pairwise ones.

Step 4 — Rounding rule

If ropa doesn't divide evenly by N (3 or 4), you can't have fractional висты. Fix: add or subtract exactly 1 очко to/from one player's ropa (by agreement) so the division is exact, and compensate the other players with a small number of висты (per §4.2.12). This is a minor correction, not a scoring event.

Step 5 — Sum up personal висты (from вистование during play)

Independently of ropa, players accumulate висты against each other for taking/missing tricks while whisting. For each pair of players, net these out: subtract the smaller from the larger to get one signed number per pair.

Step 6 — Final result per player

итог_игрока = (выигрыш/проигрыш с горы, шаг 3)
+ (сумма чистых вистов против каждого другого игрока, шаг 5)

Sum across all players should equal **zero** (preference is zero-sum) — this is the standard sanity check.

Where 3 vs 4 players matters

- N in step 3's division ($\times 10 / N$) is literally the player count.
- Rounding behavior differs slightly: with 3 players division by 3 tends to round down, with 4 players division by 4 tends to round more evenly — this is just a side effect of arithmetic, not a rule difference.
- Ленинградка explicitly notes the $\times 2$ multiplier in Step 1 applies "как при игре втроём, так и вчетвером" — same doubling regardless of player count.

One thing worth flagging for your app: the *specific* point values assigned to contracts (шестерная, семерная, мизер, etc.) and how ремизы are recorded differ between Сочинка/Ростов/Ленинградка/Классика — but the **final conversion algorithm above is the same skeleton** across all of them, just with different multipliers plugged in at Step 1. When you build the app, you'll probably want a config flag for "ruleset variant" that sets the multiplier (1 or 2) and vist rate (usually 10), and the rest of the calculation logic stays identical.